CENTRO ESTADUAL DE EDUCAÇÃO TECNOLÓGICA PAULA SOUZA

FACULDADE DE TECNOLOGIA DE BOTUCATU

Programação para Dispositivos Móveis

Guia para criação de um projeto Android no Android Studio para uso do recurso de armazenamento de dados em arquivos texto e propriedades chave - valor

Osvaldo Cesar Pinheiro de Almeida

(osvaldo.almeida@fatec.sp.gov.br)

Botucatu - SP

Outubro - 2022

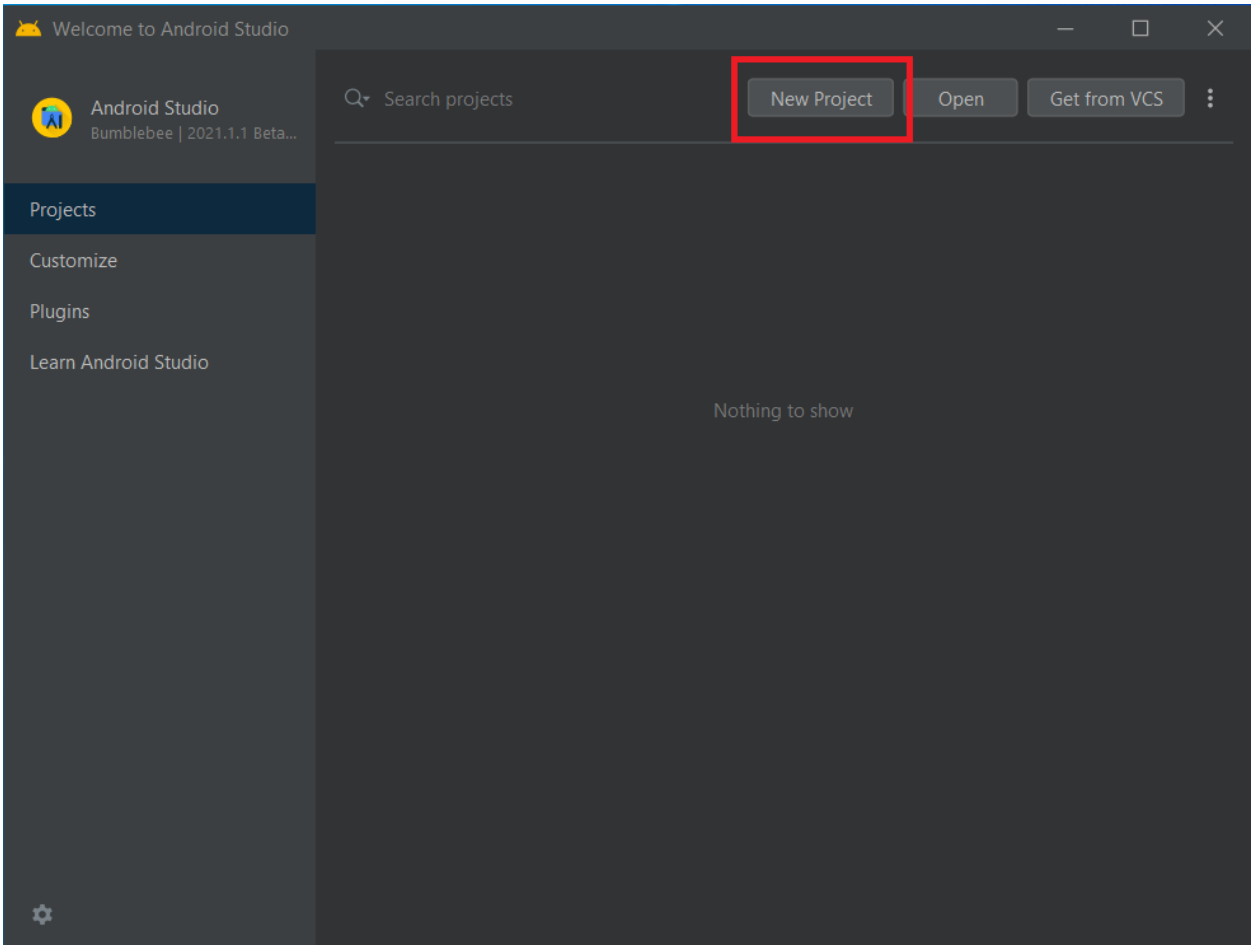
Inicialização

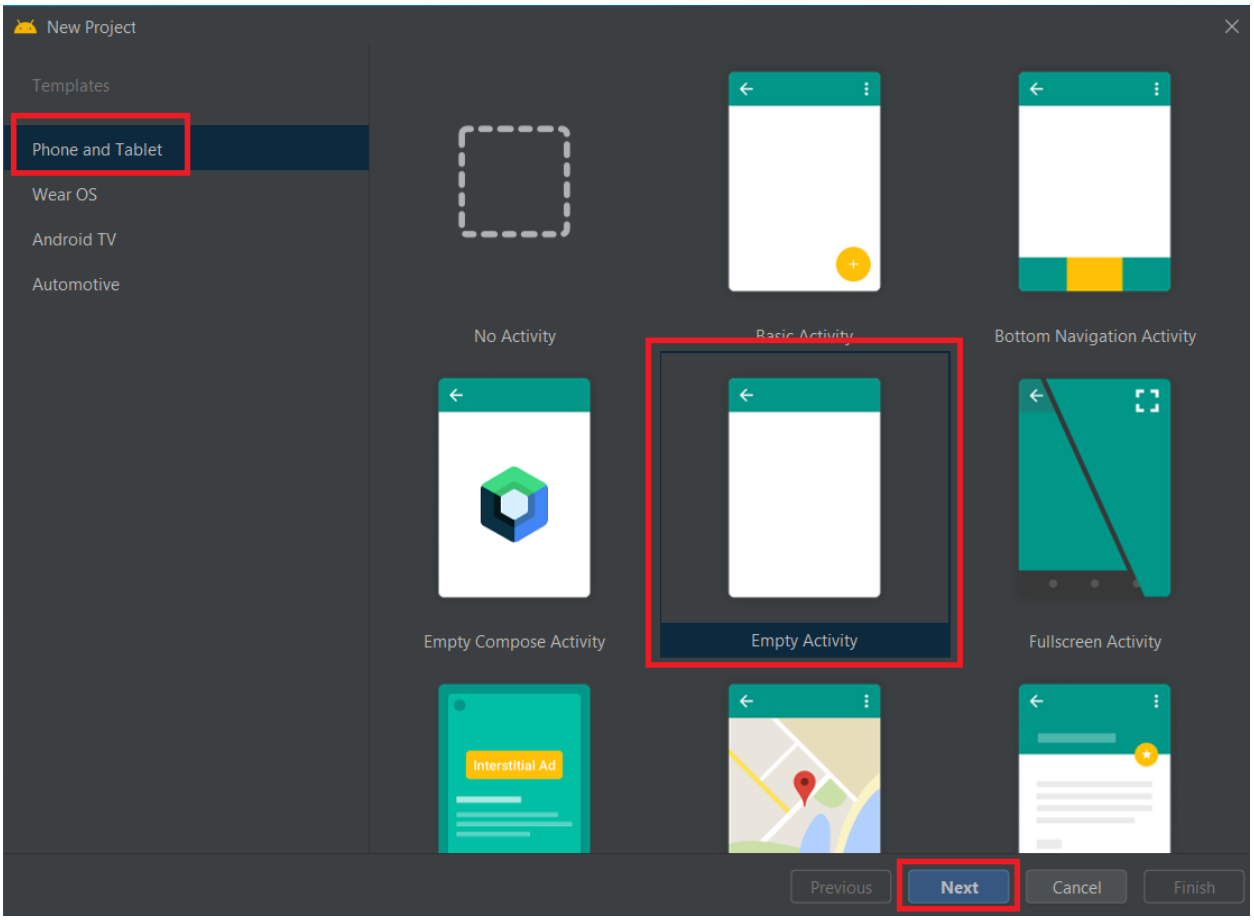
Requisitos iniciais:

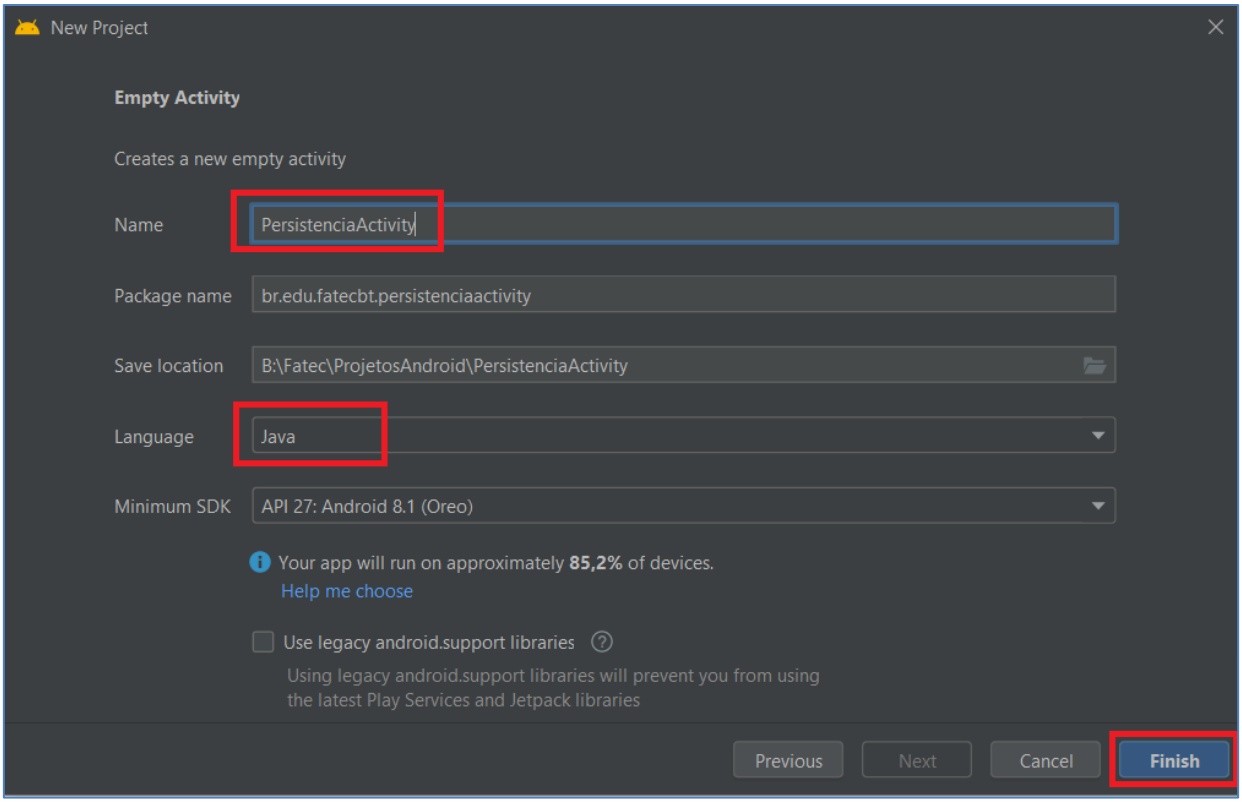
- Android Studio completo - Versão recomendada do Android 2021 ou maior;
- Observe as imagens com destaque em vermelho.

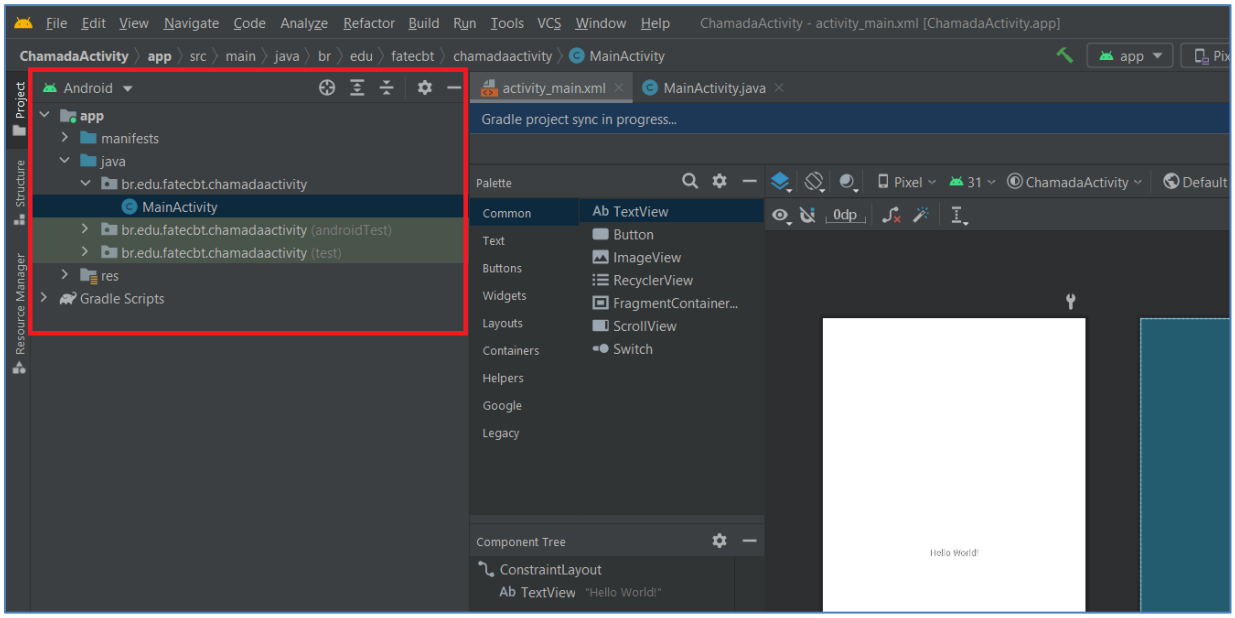
Criando Projeto

Usando o Android Studio iremos criar um novo projeto:




New Project
Templates
Phone and Tablet
Wear OS
Android TV
Automotive
No Activity
Basic Activity
Bottom Navigation Activity
Empty Compose Activity
Empty Activity
Fullscreen Activity
Interstitial Ad
Previous
Next
Cancel
Finish


New Project
Empty Activity
Creates a new empty activity
Name
PersistenciaActivity
Package name
br.edu.fatecbt.persistenciaactivity
Save location
B:\Fatec\ProjetosAndroid\PersistenciaActivity
Language
Java
Minimum SDK
API 27: Android 8.1 (Oreo)
Your app will run on approximately 85,2% of devices.
Help me choose
Use legacy android.support libraries
Using legacy android.support libraries will prevent you from using the latest Play Services and Jetpack libraries
Previous
Next
Cancel
Finish


File Edit View Navigate Code Analyze Refactor Build Run Tools VCS Window Help
ChamadaActivity - activity_main.xml [ChamadaActivity.app]
ChamadaActivity app src main java br edu fatecbt chamadaactivity MainActivity
Android
app
manifests
java
br.edu.fatecbt.chamadaactivity
MainActivity
br.edu.fatecbt.chamadaactivity (androidTest)
br.edu.fatecbt.chamadaactivity (test)
res
Gradle Scripts
activity_main.xml
MainActivity.java
Gradle project sync in progress...
Palette
Common
Text
Buttons
Widgets
Layouts
Containers
Helpers
Google
Legacy
TextView
Button
ImageView
RecyclerView
FragmentContainer...
ScrollView
Switch
Component Tree
ConstraintLayout
TextView "Hello World!"
Hello World!

Nesse projeto utilizaremos a Activity para permitir que o usuário salve informações em arquivos compartilhados e arquivos texto. No primeiro caso será utilizado o modelo de arquivamento propriedades no padrão chave-valor diretamente no arquivo compartilhado de uma Activity. Uma outra opção para armazenamento de propriedades no padrão chave-valor seria especificando um arquivo de compartilhamento, por meio da chamada do método “getSharedPreferences()”. No segundo caso será apresentado o armazenamento direto em arquivos, que poderia ser qualquer tipo de arquivo. Para ilustração dos conceitos será utilizado armazenamento de arquivos texto.

Na implementação das propriedades será realizada a inserção de dois valores para as chaves "NOME" e "PONTOS". Nessa implementação não será especificado um arquivo, portanto esses valores serão armazenados no arquivo padrão da Activity.

Para o armazenamento de dados em arquivos, o Android permite fazer o armazenamento escolhendo entre dois tipos de armazenamento: Interno e Externo. O interno permite que as informações sempre estejam disponíveis para a aplicação e quando o usuário desinstala a aplicação os dados são automaticamente deletados. No armazenamento externo o arquivo pode não estar disponível, pois depende da disponibilidade do dispositivo de armazenamento. Além disso, os dados gravados podem ser lidos em outros dispositivos e ao desinstalar a aplicação os dados gravados só serão excluídos se estiverem na pasta padrão criada pela aplicação (por meio da chamada do método “getExternalFilesDir()”).

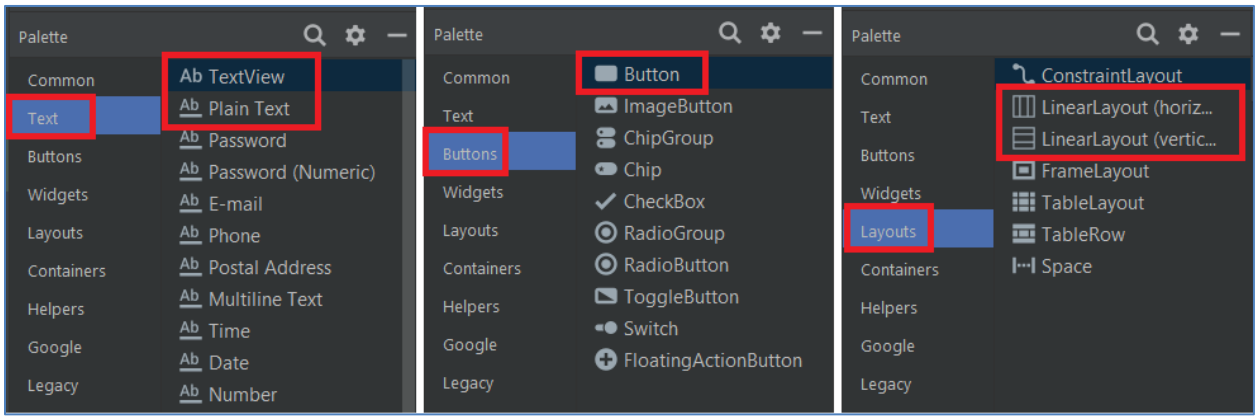
O processo para gravação usando armazenamento interno ou externo é igual. O que muda na aplicação é a definição das permissões no arquivo Manifest.xml para gravar ou ler do armazenamento externo. Além disso, é indicado que se faça algumas verificações para saber se o dispositivo de armazenamento externo (memória) está presente para ser usado.

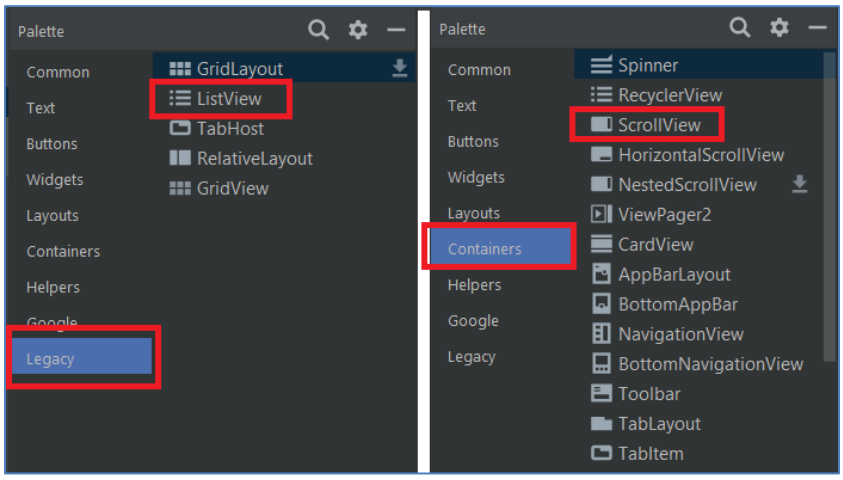
A aplicação de exemplo que iremos apresentar vai utilizar o armazenamento interno, aplicando gravações de dados em vários arquivos diferentes, a partir da escolha do usuário. Para isso, a aplicação apresentará uma lista de arquivos que o usuário deve escolher. Após a escolha de um arquivo, a aplicação irá exibir o conteúdo do arquivo e ainda permitir que o usuário entre com mais informações.

Configurando o Layout

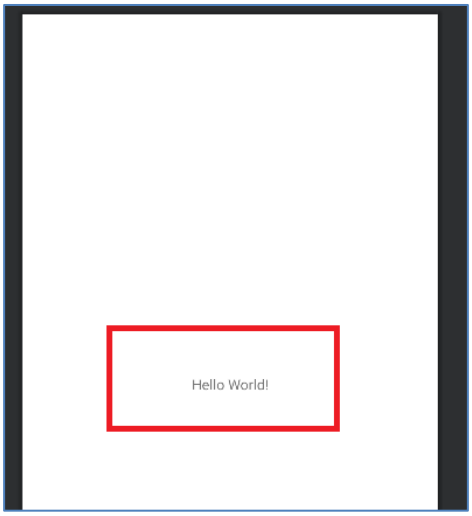
Para a criação do layout de exemplo serão usados 11 componentes gráficos, sendo 4 caixas de entrada de texto, 3 botões, 3 textos e 1 lista de visualização. Além disso, serão usados elementos de "LinearLayout" (vertical e horizontal) e um "ScrollView" para ajustar a posição dos componentes visuais.

A imagem abaixo define os elementos da paleta que serão usados:





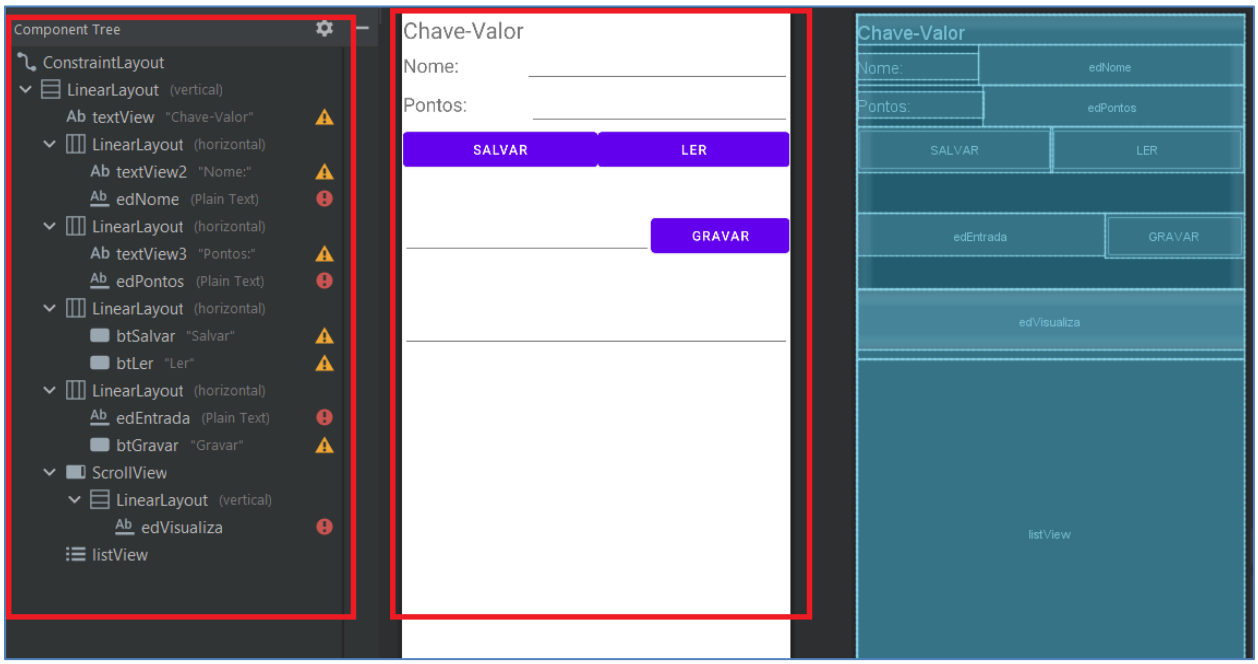
O processo de construção do layout inicia-se removendo qualquer conteúdo prévio do layout (no caso será removido o texto "Hello World").



Em seguida será adicionado o componente de posicionamento LinearLayout (vertical). Depois serão inseridos os componentes para a inserção dos valores das chaves "NOME" e "PONTOS" do arquivo padrão da Activity. Para isso iremos inserir, na seqüência: Uma TextView, depois um LinearLayout (horizontal) com uma TextView e um EditText, depois mais um LinearLayout (horizontal) com uma TextView e um Editext e, por fim, um LinearLayout (horizontal) com dois botões.

Serão inseridos na seqüência: um LinearLayout (horizontal) com um EditText (para a entrada de informações a serem armazenadas) e um botão para gravar as informações, depois um ScrollView (vertical) com um Editext (onde serão exibidas as informações dos arquivos) e, por fim, um ListView (que apresentará a lista de arquivos).

O layout após essa etapa deve ficar como a figura abaixo.



Como pode ser observado é importante mudar o "id" dos componentes para que possamos manipular os componentes na codificação. Assim, atente-se aos campos de EditText com os nomes edNome (para a chave "NOME"), edPontos (para a chave "PONTOS"), edEntrada (para a entrada das informações) e edVisualiza (para a caixa de visualização das informações). Além dos botões btSalvar e btLer, que serão usados, respectivamente, para salvar e ler os valores armazenados para as chaves, e btGravar, que será usado para gravar as informações da caixa de entrada nos arquivos de texto.

O componente EditText ("edVisualiza") deve ter o seu atributo "inputType" editado, de maneira que seja selecionado o item "textMultiLine".

Programando os processos na classe MainActivity

A programação dos processos deve ser feita na classe MainActivity. No exemplo serão utilizados quatro processos, três vinculados aos botões: btSalvar, btLer e btGravar, e outro vinculado ao clicar nos itens do "listView". A programação da classe constará das seguintes implementações:

1) Definição das constantes que irão definir o nome das chaves para "NOME" e "PONTOS", e definição de atributos que irão representar as duas caixas de entrada, o arquivo selecionado e a lista de possíveis arquivos, fazendo suas inicializações no método "onCreate";

```java
MainActivity.java
public class MainActivity extends AppCompatActivity {

    static private String KEY_NOME = "key.nome";
    static private String KEY_PONTOS = "key.pontos";

    private String[] array = new String[]{"Arquivo 1", "Arquivo 2", "Arquivo 3", "Arquivo 4", "Arquivo 5"};
    private String arquivo;
    private EditText entrada, exibicao;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        entrada = findViewById(R.id.edEntrada);
        exibicao = findViewById(R.id.edVisualiza);
    }
```

2) Criação de um método chamado "salvarChave" que irá implementar o processo para realizar a inserção dos valores paras as chaves "NOME" e "PONTOS" no arquivo padrão da Activity. Esses valores serão lidos, respectivamente, das caixas de entradas edNome e edPontos;

```java
MainActivity.java
    public void salvarChave(View view){
        SharedPreferences preferences = getPreferences(MODE_PRIVATE);
        SharedPreferences.Editor editor = preferences.edit();
        editor.putString(KEY_NOME, ((EditText)findViewById(R.id.edNome)).getText().toString());
        editor.putInt(KEY_PONTOS, Integer.parseInt(((EditText)findViewById(R.id.edPontos)).getText().toString()));
        editor.commit();
    }
}
```

3) Criação de um método chamado "lerChave" que irá implementar o processo para realizar a leitura dos valores paras as chaves "NOME" e "PONTOS" no arquivo padrão da Activity. Esses valores serão apresentados, respectivamente, das caixas de entradas edNome e edPontos;

```java
public void lerChave(View view){
    SharedPreferences preferences = getPreferences(MODE_PRIVATE);
    String nome = preferences.getString(KEY_NOME, defValue: "PADRÃO");
    int pontos = preferences.getInt(KEY_PONTOS, defValue: 0);
    ((EditText)findViewById(R.id.edNome)).setText(nome);
    ((EditText)findViewById(R.id.edPontos)).setText(String.valueOf(pontos));
}
```

4) Criação de um método chamado "exibirDados", que irá implementar o processo para realizar a leitura do arquivo texto selecionado pelo usuário e irá exibir seu conteúdo na caixa de entrada "edVisualiza". Este método deve ser vinculado à seleção de itens da "listView";

```java
private void exibirDados(){
    exibicao.setText("");
    FileInputStream in;
    try{
        in = openFileInput(arquivo);

        Scanner scanner = new Scanner(in);
        while(scanner.hasNext()){
            exibicao.append(scanner.nextLine() + "\n");
        }
    }catch (IOException ex){
        exibicao.setHint("Vazio");
    }
}
```

5) Criação de um método chamado "gravarDados", que irá implementar o processo para realizar a gravação das informações contidas na caixa de entrada no arquivo selecionado pelo usuário. Este método deve ser vinculado ao atributo "onClick" do botão "btGravar";

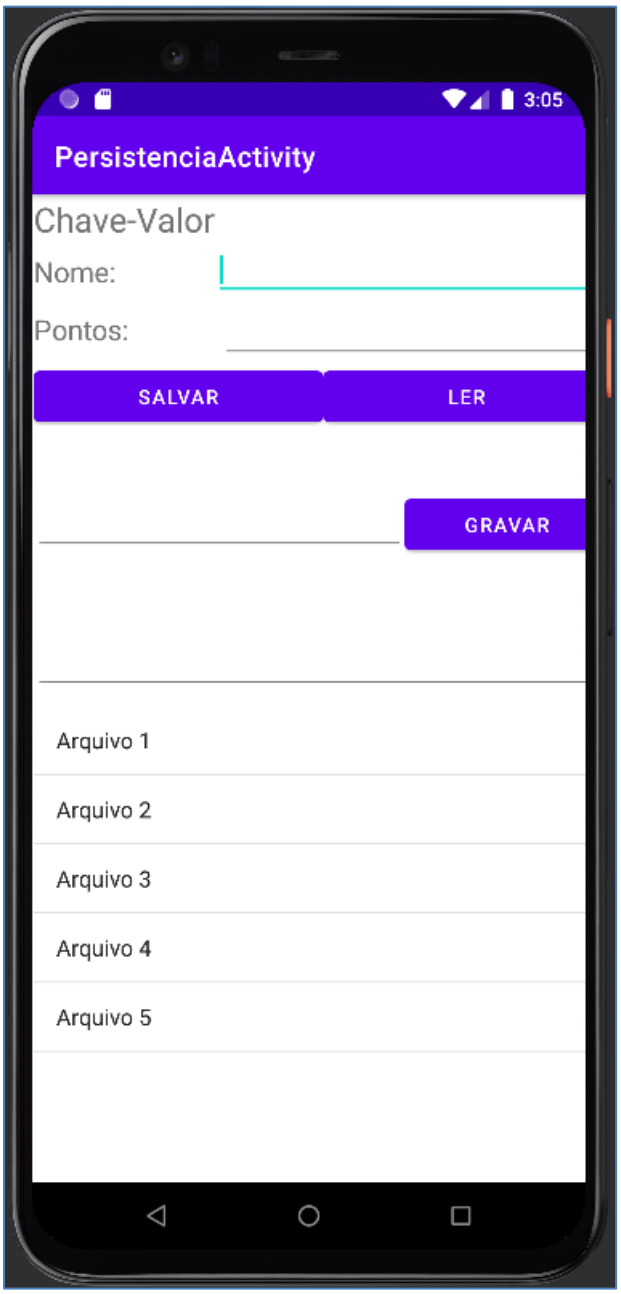
```java
MainActivity.java
public void gravarDados(View view){

    FileOutputStream out;
    try{
        out = openFileOutput(arquivo, MODE_APPEND);
        out.write(entrada.getText().toString().concat("\n").getBytes());
    } catch (IOException ex){
    }

    exibirDados();
}
```

6) Implementação do método "onCreate" para que seja realizada a vinculação de seleção dos itens da "listView" com o método "exibirDados". Isso permitirá que, ao selecionar um arquivo da lista, automaticamente sejam exibidas as informações contidas neste arquivo;

```java
MainActivity.java
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    entrada = findViewById(R.id.edEntrada);
    exibicao = findViewById(R.id.edVisualiza);

    ArrayAdapter<String> arrayAdapter = new ArrayAdapter<>(getApplicationContext(),
            android.R.layout.simple_list_item_1, array);
    ListView lista = findViewById(R.id.listView);

    lista.setAdapter(arrayAdapter);
    lista.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
            arquivo = (String) parent.getItemAtPosition(position);
            exibirDados();
        }
    });
}
```

Após a criação dos métodos é importante realizar a vinculação desses métodos com os botões, especificando o método ao atributo "onClick" do botão.

Se tudo estiver correto, basta executar o programa.



Bom trabalho!!!